

Análise de Código Java

Busca em Largura (BFS — Breadth-First Search)

1. Objetivo Geral

O código implementa o algoritmo de **Busca em Largura (BFS)** para percorrer um grafo representado como lista de adjacência (*Map<Integer, List<Integer>>*). O método **bfs()** recebe o grafo e um nó inicial, retornando uma lista com todos os nós visitados na ordem em que foram descobertos, nível por nível. O **main()** constrói um grafo de exemplo e exibe o resultado da travessia.

2. Lógica e Algoritmo

O algoritmo utiliza uma **fila (Queue)** para garantir a ordem FIFO na exploração dos nós. O nó inicial é enfileirado e marcado como visitado. A cada iteração, o nó da frente da fila é removido (*poll()*), adicionado à lista de resultado e seus vizinhos ainda não visitados são enfileirados. O truque *visited.add(neighbor)* retorna *false* se o elemento já existe no *Set*, evitando duplicatas de forma elegante e compacta em uma única linha.

Trecho central do algoritmo:

```
while (!queue.isEmpty()) { int node = queue.poll(); order.add(node); for
(int neighbor : graph.getDefault(node, List.of())) if
(visited.add(neighbor)) queue.add(neighbor); }
```

3. Conceitos Java Utilizados

O código emprega **Generics** em coleções tipadas (*List<Integer>*, *Map<Integer, List<Integer>>*), a interface **Queue** implementada por *LinkedList* para fila, e **HashSet** para verificação de visita em $O(1)$. O uso de *getOrDefault()* e *List.of()* são idiomas modernos do Java (8+) que tornam o código mais seguro e conciso, evitando *NullPointerException* em nós sem arestas.

4. Complexidade

A complexidade de tempo é $O(V + E)$, onde V é o número de vértices e E o número de arestas, pois cada nó e cada aresta são processados exatamente uma vez. A complexidade de espaço também é $O(V)$, dado que a fila e o conjunto de visitados armazenam no máximo todos os vértices do grafo.

5. Decisões de Design e Limitações

O design é limpo e funcional, mas apresenta algumas **limitações** a considerar: o grafo é tratado como **não-direcionado implicitamente** — se as arestas inversas não forem adicionadas manualmente, o grafo é, na prática, dirigido. Não há **tratamento de grafos desconexos**: nós inacessíveis a partir do nó inicial são simplesmente ignorados. Por fim, a

estrutura não suporta diretamente **pesos nas arestas** — para isso, seria necessário adaptar para Dijkstra ou BFS ponderado.